

BUT SD

SAE Apprentissage pour l'IA

Cours n°4

Architectures d'IA générative

Marc Métivier



IUT de Paris - Rives de Seine
Université Paris Cité

Plan

1. IA générative

- Méthodes actuels
- Auto-encodeurs
- GAN

2. Traitement des données textuelles

- Embeddings
- Transformers
- LLM : pre-training & fine-tuning

IA Generative

Des systèmes capables de créer du contenu nouveau (texte, image, son,...)

Modèle **Discriminatif** : apprendre $P(y|x)$

- Apprendre une distribution de probabilités sur les sorties
- Déterminer la sortie en fonction des entrées
- C'est ce que fait la classification supervisée
- Ex: Est-ce un chat ou un chien ?

Modèle **Génératif** : apprendre $P(x)$ (ou $P(x|z)$)

- Apprendre une distribution de probabilités sur les entrées
- Générer une nouvelle données à partir des distributions apprises
- Produire une donnée similaire aux données d'entrées (éventuellement selon une donnée tierce)

Familles de méthodes actuelles

Auto-encoder (AE)

- Compression dans un espace latent
- En particulier les Variational AE (VAE)

Generative Adversarial Network (GAN)

- Opposer un Générateur et un Discriminateur
- Chacun apprend à tromper l'autre

Modèle de diffusion

- Inversion d'un processus de bruit gaussien
- Génération d'image à partir du bruit
- *Stable Diffusion, Midjourney, DALL-E..*

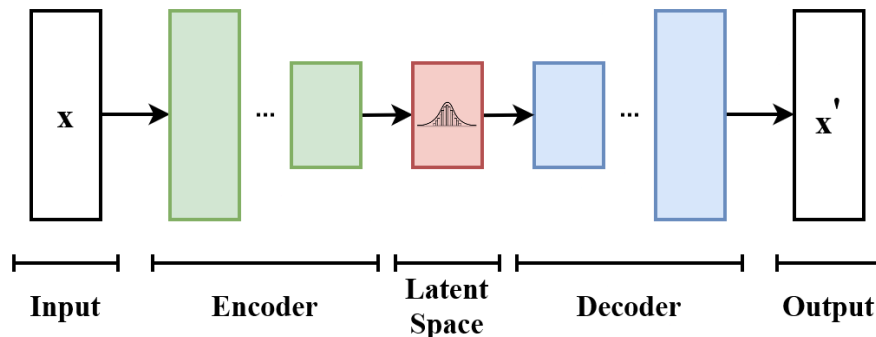
Modèle auto-régressifs (Transformer)

- Transformeurs génératifs pré-entraînés (GPT)
- *ChatGPT, Gemini, ...*

Auto-encodeurs

A l'origine, une forme d'apprentissage non supervisé

- L'encodeur extrait l'information "essentielle" et la projette dans un *espace latent*
- Le décodeur reconstruit la donnée d'entrée à partir de l'information latente
- L'espace latent doit être "petit" afin de ne stocker que l'information essentielle



EugenioTL, CC BY-SA 4.0, via Wikimedia Commons

Exemple d'usage : débruiter des données de type MNIST

- L'auto-encodeur reçoit en entrée des images bruitées
- Il produit en sortie une image "débruitées"

Réseaux antagonistes génératifs

Generative Adversarial Network (GAN) (Goodfellow et al., 2014)

Deux réseaux sont en compétition :

- **Générateur** : il fabrique des données fausses
 - Son vecteur d'entrée est aléatoire
 - Sa sortie doit produire une données dans le but de tromper le discriminateur
- **Discriminateur** : il doit reconnaître les données fausses des réelles

Un apprentissage à tour de rôle

- Tour du **discriminateur** :
 - Le générateur est figé et produit des données fausses
 - Les données fausses sont mélangées à des données réelles
 - Le discriminateur est entraîné à classer les données comme fausses ou réelles
- Tour du **générateur** :
 - Le discriminateur est figé et évalue la qualité des sorties du générateur
 - Le générateur est entraîné à produire des données trompant le discriminateur

Données textuelles et LLM

Traitement des données textuelles

Des textes comme données d'entrée

- Des données de taille variable
- Souvent de très grandes tailles
- Des données séquentielles
- Importance du contexte
 - Exemple : quel est le sens du mot "avocat" ?

Deux approches

- Réseaux de neurones récurrents (RNN)
 - Adaptés aux données séquentielles
 - Les entrées prennent en compte les sorties précédentes
 - Mais peu efficaces pour le traitement du langage
 - Séquences trop longues, phénomène d'oubli, calculs très coûteux
- Réseaux avec des mécanismes d'attention (Transformers)
 - Pas de notions de séquences mais une prise en compte du "contexte"
 - Beaucoup plus performant avec le langage naturel

Représentation numérique

Tokenization & Dictionnaire

- Découpage en jetons (tokens) : morceaux de texte séparés par un espace
 - Des mots mais aussi des combinaisons
 - Par ex : en anglais, "I've" formera un jeton
- Constitution d'un dictionnaire de jetons
 - Chaque jeton est associé à un numéro
 - Numérotation généralement arbitraire : l'ordre n'a pas d'importance

Encodage ?

- Vectorisation : la liste des numéros de jetons
 - Défaut : des valeurs numériques sans rapport avec le sens
- One Hot Encoding :
 - La phrase devient une matrice grande et creuse (sparse)
 - Défaut : la taille du dictionnaire et des données
- One Hot Vector :
 - Un vecteur binaire de la taille du dictionnaire indiquant la présence d'un jeton
 - Défaut : perte de l'ordonnancement des jetons (et leurs répétitions éventuelles)

Word Embedding

Vectorisation des mots

- Chaque **jeton** est représenté par un vecteur de **n valeurs réelles**
- Le dictionnaire est projeté dans un espace de dimension n
- Beaucoup moins volumineux que le OHE (même avec n en centaines)

Des vecteurs porteurs de sens

- La distance entre deux **jetons** reflète la proximité entre eux
- Les dimensions reflèteront des éléments de proximité
- Par ex : "Rome" et "Italie" placés d'une manière similaire à "Paris" et "France"

Deux types de construction

- **Embedding contextuel** :
 - L'embedding est porteur de sens dans le contexte de l'apprentissage actuel
 - Une simple couche dans Keras
- **Embedding global** : dictionnaires généraux, universels, développés par ML

Les *Embeddings* de Keras

La couche (layer) `Embedding`

- En entrée : un batch de N phrases de k jetons
 - Chaque phrase est une séquence de numéros de jetons
 - Padding : on ajoute des 0 jusqu'à avoir k jetons
 - Une phrase trop grande (plus de k jetons) sera tronquée
- La couche construit un dictionnaire d'embedding
 - Chaque entrée (jeton) est associé à un vecteur dense de taille n
 - Les paramètres des vecteurs denses sont les poids de la couche
 - La couche `Embedding` possède 3 paramètres principaux :
 - `input_dim` : le nombre de tokens (m)
 - `output_dim` : la taille des vecteurs denses (n)
 - `input_length` : la taille des phrases d'entrée (k)
- En sortie : un tenseur de forme (N, k, n)

Utilisés en entrée des LLM (GPT, etc..)

Transformeurs

Un type de modèle adapté aux séquences avec contexte

- Proposé en 2017 par Ashish Vaswani et al. (Google)
- Prise en compte de séquences sans besoin d'utiliser la récurrence
- Un mécanisme d'**auto-attention** pour prendre en compte le contexte

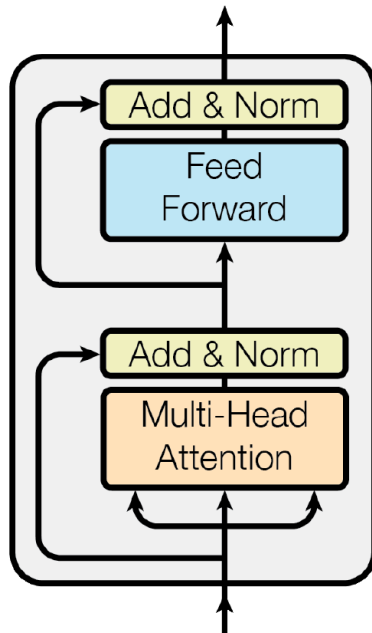
Ils ont conduit à l'émergence de grands modèles de langage (LLM)

- GPT : Transformeur pré-entraîné génératif
- BERT : Transformeur pré-entraîné non-génératif

Embedding des mots et des positions

- Les transformeurs ne sont pas des réseaux *récurrents*
 - Tous les jetons d'une séquence sont traités en même temps (donc parallèlement)
- Les jetons sont encodés par un embedding
- L'embedding des jetons est combiné à un encodage positionnel

Bloc de transformeur



Source : (Vaswani et al., 2017)

Deux parties :

- Multi-head Attention : le mécanisme d'attention
 - C'est ici que s'opère la "magie"
- Feed Forward : 1 ou 2 couches de réseau dense

Connexion résiduelle autour de chaque partie

- L'entrée d'une partie est ajoutée à sa sortie
- Le résultat est ensuite normalisée

Entrées et sorties de même forme

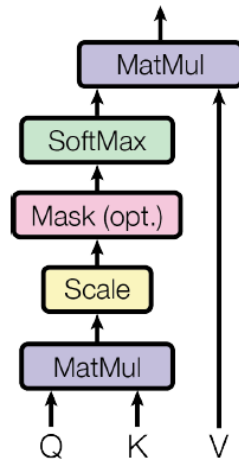
- Entrée : les vecteurs de la séquence
- Sortie : autant de vecteurs qu'en entrée
- Le contenu est "transformé" mais pas la forme

Le mécanisme d'auto-attention

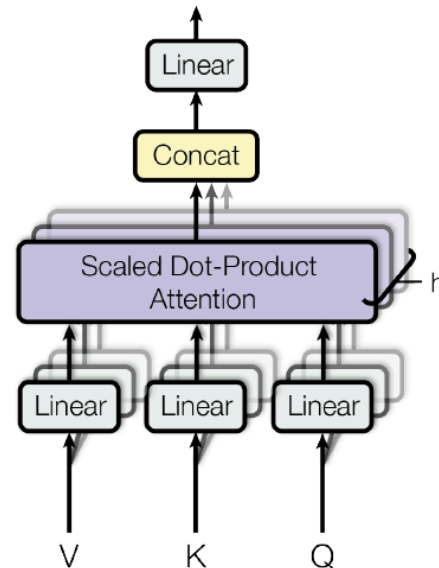
Prendre en compte les liens entre les jetons de la phrase

- Les jetons passent par 3 couches denses parallèles appelées K, Q et V
- Un score est calculé pour chaque pair de jetons
- Ce score reflète l'importance d'un jeton pour la compréhension du second
- En sortie, chaque vecteur de jeton a été modifié en conformité avec ces scores

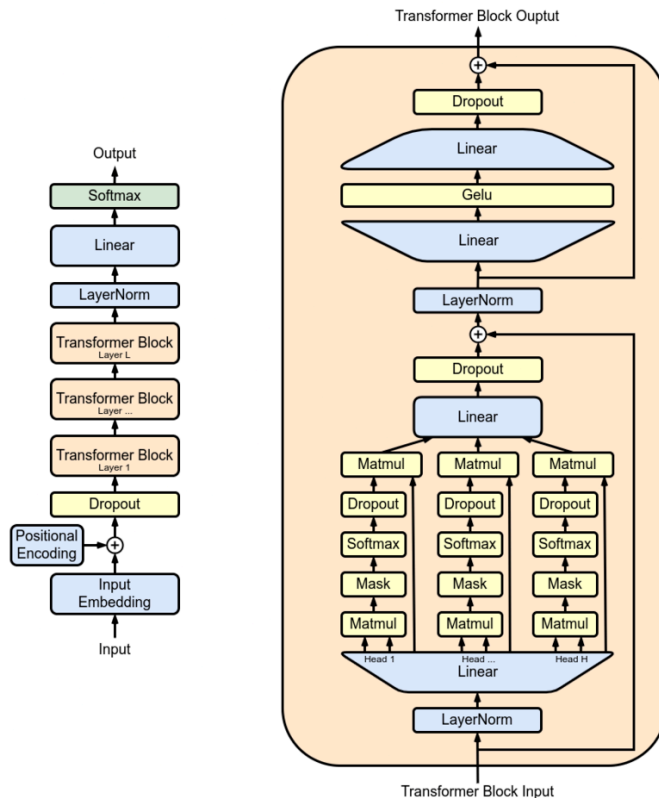
Scaled Dot-Product Attention



Multi-Head Attention



Architecture de style GPT



Generative Pretrained Transformer

- Aspect "Decodeur"
- Archi de type "Auto-regressive"
- Auto-attention avec des masques

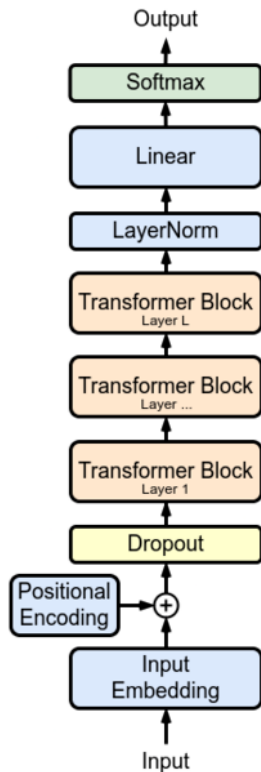
Objectif : prédire le prochain jeton

- Entrée : séquence de jetons
- Sortie : probabilités sur les jetons
 - Prédire le prochain jeton
- Séquence pour l'itération suivante
 - Premier jeton enlevé
 - Nouveau jeton ajouté à la fin

Marxav, CC0, via Wikimedia Commons

Adapté à la génération de texte

Architecture de style BERT



Marxav, CC0, via Wikimedia Commons

Bidirectional Encoder

- "Encodeur" / "Auto-encoding"
- Auto-attention sans masque

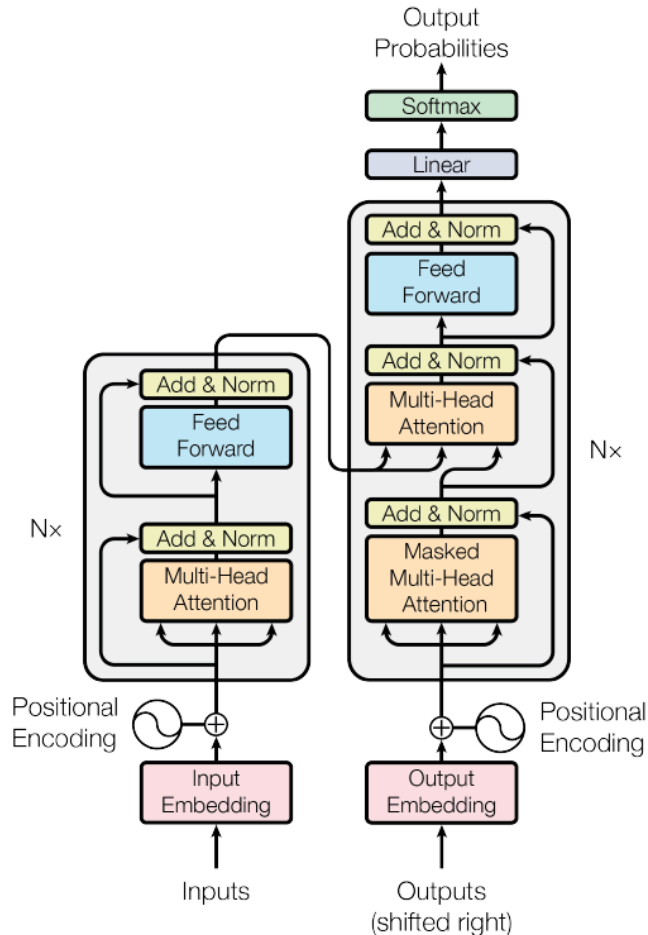
Objectif : prédire les jetons manquants

- Entrée : séquence avec des jetons manquants
 - Des jetons aléatoirement remplacés par [MASK]
- Sortie : probabilités sur les jetons
 - Prédire les jetons enlevés

Encodage global d'un langage

- Une sorte d'**embedding global** du langage

Architecture Encoder-Decoder



T5 / Sequences-to-Sequences

- Combine encodeur et decodeur

Objectif : produire une séquence

- 2 entrées de même embedding
- Sortie : probabilités sur les jetons
 - Prédire le prochain jeton
- Séquence pour l'itération suivante
 - Premier jeton enlevé
 - Nouveau jeton ajouté à la fin

Adapté à la traduction

Pre-training & Foundation Models

Modèle de langage générique

- De nombreuses applications très différentes utilisent le langage
- Il peut être utile d'avoir un modèle de langage commun en entrée

Construire un modèle de langage est très coûteux

- Enormément de données nécessaire
- Grande puissance de calcul (GPU, consommation d'énergie)
- Beaucoup de travail humain pour préparer les données
 - Nettoyage des données
 - Suppression de données indésirables, personnelles ou qui n'ont aucun sens

Entraînement en 2 parties :

- Le **pré-entraînement** sur le langage en général : très coûteux
 - Production d'un **modèle de fondation**
- Le **réglage fin** (***fine-tuning***) pour adapter le modèle à une tâche particulière
 - Beaucoup moins coûteux que le pré-entraînement !

Des modèles pré-entraînés

Il existe de nombreux LLM pré-entraînés disponibles

- Chaque modèle est associé à un *tokenizer*
- **Fine-tuning** sur une tâche spécifique :
 - On peut ajouter une couche spécifique à la tâche souhaitée et l'entraîner
 - On peut aussi ne pas ajouter de couche et le ré-entraîner sur la tâche

Quelques sites :

- Hugging Face (huggingface.co) :
 - La plus grande BD de modèles de transformeurs libres
 - La règle est de publier tout : l'archi, les poids, etc
- Kaggle (www.kaggle.com)
- Keras Hub : des modèles ouverts sont disponibles
 - BERT, DistilBERT, Gemma 2B/7B, GPT-2, Mistral 7B, ...
 - Parfois avec d'autres langues que l'anglais

Prompting

Piloter le modèle de langage à partir de textes rédigés en langage naturel

- Pouvoir guider le genre de réponses attendus :
 - Questions/réponses
 - Résumé
 - Raisonnement logique
- Des "bons" prompts et des "mauvais" prompts

Un domaine en plein essor

- Le but est d'améliorer la façon dont les modèles génèrent des réponses

